

MORROW

Complete User Manual

Private intelligence, built around you.

Version 0.1.0-beta.31

July 2026

Product	Morrow — local-first personal AI agent
Status	Early Access (Windows 10/11 x64; Linux source build)
Owner	Aidan Magee — aidan@getaxiom.ca
Organization	Axiom International · getaxiom.ca
Website	https://morrowproject.getaxiom.ca

This manual documents the product as verified in the repository at the beta.31 release. Feature claims are backed by tests or documented evidence; known limitations are called out explicitly.

Table of Contents

1. Introduction	5
1.1 What Morrow is for	5
1.2 Product principles	5
2. Architecture overview	6
2.1 System view	6
2.2 Repository map	6
2.3 Trust boundaries	7
3. Installation	7
3.1 Windows quick install	7
3.2 Verify or troubleshoot	7
3.3 Upgrade and rollback	8
3.4 Uninstall	8
3.5 Linux source build	8
4. First run and onboarding	8
4.1 Surfaces at a glance	9
5. The command line	9
5.1 Core commands	9
5.2 Mission Control (interactive)	9
5.3 Model picker and live status (beta.30+)	10
6. Missions — accountable work	10
6.1 Mission lifecycle	10
6.2 Success criteria	11
6.3 Evidence ledger	11
6.4 Checkpoints and rollback	11
6.5 Independent review and honest grading	11
6.6 Durability and resume	12
6.7 Failure intelligence	12

7. Providers, presets, and routing	12
7.1 Capability matrix	12
7.2 Configuring a provider	12
7.3 Credential reference	13
7.4 Presets	13
7.5 Honest OAuth findings	14
8. Tools and execution	14
8.1 Read-only tools and containment	14
8.2 Write and terminal tools	15
8.3 Browser and vision (beta.31)	15
8.4 Approval and cancellation	15
9. Privacy and security	16
9.1 Privacy modes	16
9.2 Required safeguards	16
9.3 Data categories and retention	16
10. Memory and context management	17
10.1 Memory service	17
10.2 Context management	17
10.3 Symbol index	17
11. Troubleshooting and support	17
11.1 First-line diagnostics	17
11.2 Common situations	18
11.3 Crash recovery	18
12. Known limitations and roadmap	18
12.1 Current alpha / beta limitations	18
12.2 Roadmap stages	19
13. Glossary	19
13.1 Ownership and licensing	19
13.2 Reporting issues	20

1. Introduction

Morrow is a self-hosted, deeply customizable personal AI agent. It is local-first, provider-neutral, and built around you: conversations, project state, memory, provider credentials, logs, and diagnostics remain on your machine unless a tool or configured model provider is explicitly used. Morrow is designed for visible execution, explicit permissions, and persistent memory — and it grades its own work honestly rather than asking you to trust a model's self-assessment.

1.1 What Morrow is for

Morrow helps you run accountable work through a supervised agent. A **mission** defines success up front, executes under supervision, records what happened, verifies the outcome with concrete evidence, obtains an independent review, and grades itself honestly. Every part of it is durable — a service restart never loses the objective, criteria, evidence, checkpoints, failures, or reviewer verdict.

- Natural chat with project-aware memory.
- Terminal, filesystem, browser, web, vision, voice, and coding tools.
- Local and cloud model providers with intelligent routing.
- Persistent named agents with separate roles, memory, tools, and permissions.
- Scheduled tasks, triggers, webhooks, and messaging integrations.
- Skills, plugins, MCP servers, and Hermes-compatible imports.
- Detailed privacy, cost, execution, and verification records.

1.2 Product principles

Principle	What it means in practice
Simple by default, powerful by choice	New users get polished presets; advanced users can customize models, tools, memory, agents, permissions, workflows, and interface behavior.
Local-first and provider-neutral	You own your data and can choose local models, cloud models, or a controlled mix.
Visible execution	Plans, tool calls, files, costs, external data sharing, and agent activity are inspectable.
Reliable autonomy	Long-running tasks persist, recover, retry safely, and verify their own results.
Reversible actions	File changes, configuration, memory, and automations support history and rollback.
Proof over claims	Morrow publishes repeatable parity and performance tests instead of relying on marketing language.

! Status note

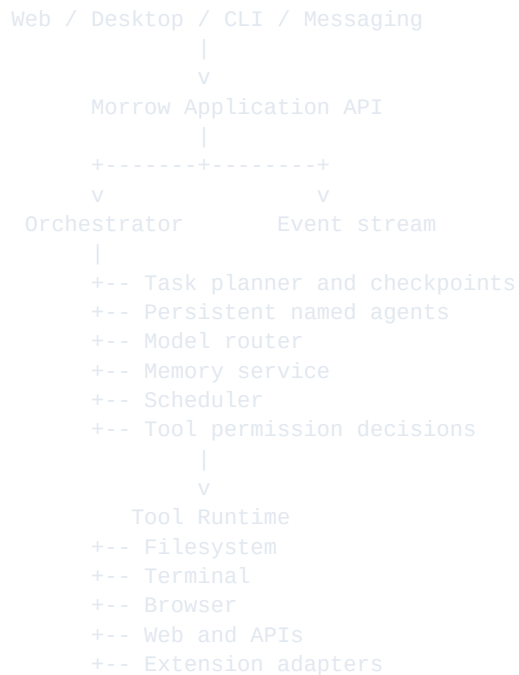
Morrow v0.1.0-beta.31 is an Early Access release. Windows 10/11 x64 is supported; Linux is source-build only; macOS is planned. Some capabilities (write tools, subscription OAuth for some providers, live model discovery) are intentionally gated. See Section 12 for the full list of current limitations.

2. Architecture overview

Morrow is a monorepo (pnpm workspaces + Turbo) with a clear separation between the client surfaces, the orchestrator that owns tasks and execution, and the provider/runtime boundary. The browser client is never trusted with provider secrets, and models are never trusted to grant themselves permissions.

2.1 System view

Web / Desktop / CLI / Messaging clients talk to the Morrow Application API, which fronts the Orchestrator. The Orchestrator owns the task planner, persistent named agents, the model router, the memory service, the scheduler, and tool-permission decisions. Below it, the Tool Runtime executes filesystem, terminal, browser, web, and extension actions.



2.2 Repository map

Path	Role
apps/web	Primary product interface: navigation, conversation, projects, settings, customization studio.
apps/cli	The `morrow` CLI plus interactive terminal / TUI (Mission Control).

Path	Role
services/orchestrator	Tasks, plans, agents, scheduling, provider runtime, persistence (SQLite).
services/runtime	Model execution and tool invocation behind explicit contracts (scaffold).
packages/contracts	Canonical Zod schemas / protocol types shared across packages.
packages/hermes-compatible	Narrow compatibility layer for importing supported Hermes config, skills, memory, sessions.
packages/ui, packages/config	Design system / shared config (README-only scaffolds in beta.31).
apps/desktop	Native desktop shell (scaffold).

2.3 Trust boundaries

- The browser client is not trusted with provider secrets.
- Models are not trusted to grant themselves permissions.
- Tool results are treated as untrusted input.
- Extensions run with declared capabilities only.
- External model providers receive only the context selected for that request.
- Provider credentials and tool authority do not live in the web client.

3. Installation

The Windows installer bundles its own Node.js runtime, so Git, Node.js, and pnpm are not required for the Windows install. It verifies the artifact's SHA-256 checksum, stages and validates the package, preserves existing user data during upgrades, adds `morrow` to your user PATH, and starts the loopback service. It does not open a browser.

3.1 Windows quick install

```
# In PowerShell:
iex (irm https://morrowproject.getaxiom.ca/install.ps1)

# After install, open a NEW PowerShell window:
morrow          # opens the terminal agent shell
morrow onboard  # guided provider setup
```

Provider credentials are saved in Morrow's local owner-readable secrets file and are not written to task events, reports, diagnostic exports, or browser storage.

3.2 Verify or troubleshoot

```
morrow --version
morrow status
morrow doctor          # consumer-readable health checks
morrow doctor --json    # one stable JSON document for automation
```

```
morrow doctor --export # redacted diagnostic bundle
```

• Unicode in PowerShell

If PowerShell shows broken Unicode glyphs, use Windows Terminal with a UTF-8 profile and rerun ``morrow doctor``. Plain-text and redirected output remain available when terminal capabilities are limited.

3.3 Upgrade and rollback

Rerun the quick-install command to upgrade. The installer replaces only the application tree under `%LOCALAPPDATA%\Morrow\app`; conversations, memory, configuration, provider credentials, logs, cache, and backups are preserved. The previous application tree remains available until the new service passes an identity-checked health probe, and a failed activation is rolled back.

3.4 Uninstall

```
morrow uninstall # interactive; keeps local data
morrow uninstall --yes --keep-data
morrow uninstall --yes --purge-data # PERMANENTLY deletes all local data
```

✗ Destructive

``--purge-data`` permanently deletes local conversations, memory, project state, configuration, provider credentials, backups, logs, and cache. It cannot be undone.

3.5 Linux source build

Linux is source-build only; macOS is not supported in beta.31. Requirements: Node.js 22+ and pnpm 10.x.

```
git clone https://github.com/Mageester/morrow.git
cd morrow
pnpm install
pnpm check
pnpm test
pnpm build
pnpm --filter @morrow/cli morrow
```

4. First run and onboarding

On a fresh install the fastest path is the guided onboarding, which connects a provider and creates your first project workspace. You can use either the CLI or the web interface.

- 1 Run `morrow onboard` (CLI) or open the Onboarding Wizard in the web app.
- 2 Connect at least one model provider: paste an API key (app) or run `morrow providers configure <provider> --key <KEY>` (CLI).
- 3 Pick a default model and, optionally, a preset such as Balanced or Private Local.
- 4 Create a project pointed at a workspace (folder / repository) path.
- 5 Open a conversation and send your first message.

4.1 Surfaces at a glance

Surface	Entry point	Notes
CLI / TUI	morrow <command>; interactive terminal	Mission Control is the primary live task surface.
Web	apps/web (React/Vite)	MissionControl, OnboardingWizard, ProviderManager, SkillsControlCenter, SystemHealth.
Backend	services/orchestrator	Fastify server, default 127.0.0.1:4317. CLI/web are clients of its HTTP + SSE API.

5. The command line

The morrow CLI is the most direct way to interact with the agent. The commands below are the ones you will use day to day.

5.1 Core commands

Command	Purpose
morrow	Open the interactive terminal agent shell.
morrow onboard	Guided provider + project setup.
morrow status	Show service / environment health.
morrow doctor [--json --export]	Consumer-readable (or machine-readable) diagnostics.
morrow providers configure <p> --key <K> [--url <u>] [--model <m>]	Add/telemetry-free configure a provider credential.
morrow providers test <p>	Verify a provider connection.
morrow providers remove <p>	Delete a stored credential.
morrow providers	List configured providers and status.
morrow mission "<objective>"	Start an accountable mission (see Section 6).
morrow mission list show result criteria evidence failures checkpoints [id]	Inspect missions.
morrow update	Check for a newer version (pre-release aware).
morrow uninstall	Remove the application (keeps or purges data).

5.2 Mission Control (interactive)

Inside the interactive shell, `morrow mission` opens the primary Mission Control session — the same live task surface as chat/fix, with streaming events, approvals, and Ctrl+C cancellation. The following slash commands are available:

Command	What it shows
/model	Interactive model picker with live status / context.
/status	Premium live-status experience (current request vs cumulative session).
/context	Canonical context / token usage (agrees with /status).
/output	Rendered task output.
/diff	Change applied to files (GET /api/tasks/:id/diff).
/undo	Roll back the change (GET /api/tasks/:id/undo).
/tree	Persisted task / subagent tree (GET /api/tasks/:id/tree).
/result	Mission aggregate: status, provider/model, plan, files, evidence, approvals, next safe action.
/criteria /evidence /failures /checkpoints	Mission detail views.
/panic	Stop all running work.

5.3 Model picker and live status (beta.30+)

Beta.30 added an interactive `/model` picker with honest live status/context, and a normalized reasoning-effort control that is wired through to the provider request. The picker groups models by provider, tags the provider default, honestly marks unconfigured rows, and exposes a detail panel (endpoint, context window, usable input, output reserve, tool/vision support, pricing, configuration state). The live status distinguishes the **current request** from the **cumulative session** and never fabricates a cost split it cannot prove.

6. Missions — accountable work

A **mission** is Morrow's unit of accountable work. Instead of trusting that an agent finished, a mission defines success up front, executes under supervision, records what happened, verifies the outcome with concrete evidence, obtains an independent review, and grades itself honestly.

```
# From inside a repository (or with a selected project):
morrow mission "Find and fix the most important runtime bugs in this project.
Preserve intended behaviour and prove the repaired application works."
```

6.1 Mission lifecycle

- 1 Morrow drafts measurable **success criteria** and shows you the mission contract.
- 2 It waits for approval (or auto-approves in autonomous / --yes / yolo runs, still displaying and persisting the contract).
- 3 It takes a **checkpoint** before making changes.
- 4 It **executes** the work with the agent.

- 5 It **verifies** each criterion with real evidence (command exit code, HTTP probe, bounded diff check, ...).
- 6 It sends the result to an **independent reviewer** (a separate execution).
- 7 It **grades** the mission honestly and prints the result.

6.2 Success criteria

Criteria are measurable and testable. Vague ones (“make it better”, “ensure quality”) are rejected and rewritten into observable outcomes. Each criterion has a stable id, description, state, a verification strategy, references to the evidence that proves it, optional failure/waiver reasons, and timestamps.

State	Meaning
proposed → approved → in_progress → verified	The healthy progression of a criterion.
failed / waived / unverified	Reached when a criterion cannot be proven or is explicitly waived.

Verification kinds: command, test, build, typecheck, lint, runtime, http, browser, diff, review, manual, artifact.

6.3 Evidence ledger

A criterion becomes **verified** only when it is connected to evidence whose status is passed — never because an agent said so. Evidence records the type, a concise summary, the command and exit code (when applicable), a reference to the full output, and the status (passed / failed / inconclusive). A command that passes for one criterion does not silently prove another — evidence is explicitly linked to the criterion it verifies. The ledger is persisted and viewable after the mission completes.

6.4 Checkpoints and rollback

Before risky changes Morrow captures a **checkpoint** that snapshots the exact content of the affected files (git HEAD is recorded for reference). **Rollback** restores only those captured files — it never blanket-resets the working tree and never touches unrelated pre-existing work. Rollback explains what it will change, fails safely if a needed snapshot is missing, and works after a service restart because snapshots live on disk.

6.5 Independent review and honest grading

After primary verification the mission transitions to *reviewing* and a **separate** reviewer execution runs with isolated instructions — given the objective, approved criteria, the diff, the evidence ledger, and unresolved failures, but **not** the implementing agent's narrative. Verdicts: **approved**, **approved_with_risks**, **revisions_required**, **insufficient_evidence** (which can never become a full success).

Final status	Condition
completed	All criteria verified and the reviewer approved.
completed_with_reservations	All accounted for, but something was waived or a risk was flagged (or no independent approval on record).
partially_completed	Some criteria failed or remain unverified.

Final status	Condition
blocked	Safe automated recovery was exhausted / no criteria could be set.
failed	Nothing could be proven.
cancelled	Cancelled by the user.

6.6 Durability and resume

Mission state is stored in SQLite (missions, mission_criteria, mission_evidence, mission_failures, mission_checkpoints, mission_reviews, and an append-only mission_events timeline). A restart reconstructs the mission entirely from persistence — `morrow mission show` and the slash commands work across restarts. Only concise decisions, actions, evidence, and summaries are stored; raw model reasoning is never persisted or displayed.

6.7 Failure intelligence

Every meaningful failure is persisted with a category (patch-context mismatch, test/build failure, provider failure, timeout, ...) and a normalized signature that collapses volatile detail so repeats are detected. Recovery escalates deterministically, and the same failed operation is never repeated forever. When safe automated options are exhausted, the mission escalates to **blocked** rather than spinning.

7. Providers, presets, and routing

Morrow runs every model provider through one provider-neutral runtime. All adapters normalize to the same streaming chunk shape and the same typed error classification (auth, rate_limit, timeout, network, cancelled, invalid_request, provider).

7.1 Capability matrix

Provider	Kind	Stream	Tools	Vision	Local
OpenAI	api-key	yes	yes	yes	—
Anthropic	api-key	yes	yes	yes	—
Google Gemini	api-key	yes	yes	yes	—
OpenRouter	api-key	yes	yes	yes	—
DeepSeek	api-key	yes	yes	no	—
OpenAI-compatible	api-key	yes	yes	no	—
Ollama	local	yes	yes	no	yes

7.2 Configuring a provider

There are three ways to give Morrow a provider credential. None require PowerShell, manually setting environment variables, or restarting the service.

- 1 In the app (recommended).** Settings → Providers → Configure. Paste the API key, optionally set a custom endpoint and default model, then Save. The key is sent once to the local orchestrator, persisted to the secrets file, and applied immediately. Test connection verifies it.
- 2 From the CLI.** `morrow providers configure <provider> --key <KEY>` (optionally `--url` and `--model`). Same running-service endpoint, so it also takes effect with no restart.
- 3 Pre-seeded environment.** Env vars set in the shell before the service starts are honored. A shell-set variable takes precedence over a saved one; the app and CLI warn you when that shadowing happens.

✓ Where keys live

Keys are stored server-side in the owner-readable secrets file and never reach the browser (no localStorage), the database, logs, errors, or task events. Provider status exposes only `configured`, the default model, and the endpoint *host*.

7.3 Credential reference

Provider	API key env	Base URL env	Default endpoint
OpenAI	OPENAI_API_KEY	OPENAI_BASE_URL	https://api.openai.com/v1
Anthropic	ANTHROPIC_API_KEY	ANTHROPIC_BASE_URL	https://api.anthropic.com
Gemini	GEMINI_API_KEY / GOOGLE_API_KEY	GEMINI_BASE_URL	generativelanguage.googleapis.com
OpenRouter	OPENROUTER_API_KEY	OPENROUTER_BASE_URL	https://openrouter.ai/api/v1
DeepSeek	DEEPSEEK_API_KEY	DEEPSEEK_BASE_URL	https://api.deepseek.com/v1
OpenAI-compatible	OPENAI_COMPAT_API_KEY (opt)	OPENAI_COMPAT_BASE_URL (req)	— (OPENAI_COMPAT_MODEL)
Ollama (local)	—	OLLAMA_BASE_URL (req to enable)	http://127.0.0.1:11434/v1

7.4 Presets

Each preset is a routing policy with concrete budgets. The router picks the first configured provider in `providerOrder`, resolves a model preference, and reports the decision (including fallbacks and the candidates considered).

Preset	Provider order (first few)	Privacy	Notes
Best Quality	anthropic → openai → gemini	cloud	Frontier models, quality first
Balanced	openai → anthropic → gemini	cloud	Sensible default

Preset	Provider order (first few)	Privacy	Notes
Fast	openai → gemini → deepseek	cloud	Low latency
Cheap	deepseek → gemini → openai	cloud	Lowest hosted cost
Coding	anthropic → openai → deepseek	cloud	Low temperature, more tool turns
Research	gemini → anthropic → openai	cloud	Large-context synthesis
Private Local	ollama	local-only	Never leaves the machine; requires Ollama

✓ Local-only guarantee

`Private Local` is local-only: it will not route to a hosted provider, even on an explicit override.

7.5 Honest OAuth findings

Morrow does **not** reverse-engineer private authentication, read browser cookies, or reuse an existing browser session. Subscription sign-in goes through each provider's real OAuth endpoints using the same first-party OAuth client ids and PKCE flow the official CLIs use, behind an explicit security/ToS warning, with tokens stored locally.

Flow	Status	Finding
Codex / ChatGPT (OpenAI)	Available	Subscription sign-in via the Codex CLI's first-party OAuth client + PKCE. Tokens target OpenAI's Codex backend. Stored locally.
Claude (Anthropic)	Available	Subscription sign-in via Claude Code's first-party OAuth client + PKCE. Subscription inference is for Anthropic's own tools and may be rejected. Stored locally.
Gemini (Google)	Unavailable	Documented Generative Language API uses API keys; Google OAuth applies to Cloud/Vertex, not consumer-subscription third-party sign-in.

8. Tools and execution

Morrow exposes tools behind explicit, inspectable boundaries. Read-only tools run behind a shared containment layer; write and terminal tools are gated behind approval; the browser is connected progressively and only after a durable approval for the exact origin.

8.1 Read-only tools and containment

The read-only tools — `inspect_workspace`, `list_files`, `read_file`, `search_files` — sit behind a shared containment layer that rejects path traversal, symlink escapes, secrets, and binary files, enforces byte and depth limits, and records evidence for every read.

- **Inspect workspace is deterministic**

The Inspect-workspace task has no network access, no model invocation, and no shell execution. It operates entirely locally and predictably. If the orchestrator restarts while tasks run, interrupted tasks are recovered and transitioned to a safe `interrupted` state.

8.2 Write and terminal tools

Write and terminal tools are intentionally not enabled by default in beta.31; the architecture and UI are sketched but gated until their full safety boundaries are implemented. When enabled, a proposed command or patch creates a pending approval record and the task waits (`waiting_for_approval`). Categorically-dangerous actions are blocked **before** an approval is even created — YOLO cannot bypass this.

8.3 Browser and vision (beta.31)

Beta.31 connects Morrow's hardened Playwright controller to the durable agent runtime. Browser tools are exposed progressively for browser/frontend requests so unrelated coding turns do not pay the context cost.

- `browser_open` creates a task-scoped session only after a durable approval for the exact HTTP(S) origin.
- `Snapshot`, `console`, `click`, `fill`, `key`, `select`, `viewport`, `screenshot`, `download`, and `close` use the approved session; the session closes on every task exit path.
- Screenshots/downloads live under `MORROW_HOME/artifacts/browser/<task-id>`; screenshot evidence records route, viewport, byte size, SHA-256, and whether vision attachment was allowed.
- PNG bytes are attached ephemerally only when the selected model has positive vision metadata; base64 is never stored in conversation text, tool output, events, or audit records.
- A frontend change requires an approved navigation, explicit DOM snapshot, console/page-error inspection, at least one interaction, and vision-attached screenshots at 1440×900, 768×1024, and 390×844.

✗ Browser safety

URL credentials and non-HTTP(S) schemes are rejected. Credential, token, secret, payment, purchase, transfer, destructive account, release, deploy, publish, and push interactions are categorically blocked from autonomous browser actions. Upload/download containment rejects symlink escapes.

8.4 Approval and cancellation

- Approval: agent proposes a command/patch → a pending approval record is created → user approves/denies → the saved tool call resumes.
- Cancellation: `POST /api/tasks/:id/cancel` aborts the `AbortController`, propagates to descendants, and flips queued/running work to `cancelled`.
- Duplicate cancellation is idempotent; already-terminal tasks return a structured 409; a late approval cannot revive cancelled work.

- Windows command cancellation uses `taskkill /F /T /PID` and is verified with a parent/child/grandchild process tree plus an unrelated survivor process.
- `/panic` stops all running work.

9. Privacy and security

Privacy is a user-visible product behavior, not only a backend policy. For every model request or tool action, Morrow can explain which model/service is involved, whether it is local or remote, what conversation content is included, which files and memories are included, which credentials are used (without exposing their values), which network destinations are permitted, and what will be retained afterward.

9.1 Privacy modes

Mode	Behavior
Local only	Local storage + local inference. No external providers, tools, or telemetry. Network-deny tests must pass.
Controlled cloud	User-approved providers; request-by-request context disclosure; external destinations recorded; provider fallback cannot silently change privacy behavior.
Custom	Per-project / per-agent rules, domain allowlists, model restrictions, retention controls, explicit exceptions.

9.2 Required safeguards

- No silent telemetry.
- No secret values in prompts or logs.
- No cross-project memory retrieval.
- No external provider fallback without disclosure.
- No plugin or skill access beyond declared capabilities.
- Automatic memory stores concise normalized conclusions and evidence references — never raw chain-of-thought; secret-like and prompt-poisoned candidates are rejected before admission.
- Automatically learned skills are project-scoped, need two distinct successful mission observations, and cannot request network access or secrets; invalid bundles are quarantined.
- Complete deletion for user-requested local data removal.
- Provider continuation fields needed for protocol correctness are access-restricted with task state, excluded from public events, logs, summaries, search, exports, and APIs, and never presented as model reasoning.

9.3 Data categories and retention

Data categories: conversation data, personal memory, project memory, agent memory, files/repository content, credentials and secret handles, tool outputs, usage and cost records, and execution history. Each requires an

explicit scope and retention rule.

Mission continuity retention: structured checkpoints contain concise decisions and execution facts, never hidden chain-of-thought. Full raw conversation, tool, and event records remain authoritative and are not destroyed by provider compaction. Deleting a task cascades its execution segments, provider turns, checkpoints, private continuation rows, and canonical answer.

10. Memory and context management

10.1 Memory service

Morrow uses a deterministic, project-isolated, user-controlled SQLite memory layer — no hidden capture, no cross-project leakage. Automatic memory retrieval increments a local usage counter so influence remains auditable, and is limited to the current project/conversation scope. Expired, stale, invalidated, retired, disabled, and candidate records are excluded.

In beta.31, automatic Cortex memory builds/refreshes at mission creation, captures only deterministic repository facts and evidence-backed mission learnings, and injects ranked active memory into later matching work without manual save/refresh/index commands.

10.2 Context management

Agent model requests pass through the local context manager before provider execution. The manager resolves model-aware budgets, counts tokens with exact offline tokenizers where available (labeled conservative estimates elsewhere), preserves system instructions and tool-call groups, compacts older eligible history into redacted persisted summaries, and refuses provider calls when the minimum viable prompt cannot fit.

Durable mission execution is segmented without replacing the task, mission, event, provider, or execution-kernel boundaries. Migration 32 adds execution segments, discrete provider turns, structured checkpoints, private provider continuations, and canonical task answers. A checkpoint, compaction, route change, restart, or turn-budget rollover cannot mark a task or mission complete.

10.3 Symbol index

Project code intelligence uses a local symbol index rather than sending whole repositories to a model. The orchestrator scans only inside the registered project root, applies .gitignore / .morrowignore / dependency-build-cache ignores and secret-like path denial, then persists symbol metadata and parser diagnostics in SQLite. TS/JS/TSX/JSX symbols are extracted with the TypeScript compiler API; agent access is read-only through concise symbol locations.

11. Troubleshooting and support

11.1 First-line diagnostics

```
morrow --version
morrow status
morrow doctor          # consumer-readable
```

```
morrow doctor --json # automation
morrow doctor --export # redacted diagnostic bundle
```

`morrow doctor` does not start the service implicitly. A stopped or unhealthy service is reported with remediation and a non-zero exit code. JSON mode writes one stable JSON document to stdout. Diagnostic exports redact secret fields, credential-shaped values, and the user-home prefix.

11.2 Common situations

Situation	What to do
Broken Unicode glyphs in PowerShell	Use Windows Terminal with a UTF-8 profile; rerun <code>`morrow doctor`</code> .
Health-failure rollback concern	The installer performs an atomic, data-preserving swap; a corrupt package rolls back with data intact (covered by tests).
Stuck task / cancellation	Use <code>/panic</code> to stop all, or <code>POST /api/tasks/:id/cancel</code> . Late approvals cannot revive cancelled work.
Restart mid-task	Recovery transitions <code>running→interrupted</code> and re-dispatches orphaned queued tasks; <code>`morrow mission show`</code> works across restarts.
Need a support bundle	Use <code>`morrow doctor --export`</code> for a redacted diagnostic bundle (logs + versions + task state).

11.3 Crash recovery

On boot, the orchestrator recovers running tasks (`running→interrupted`), interrupts streaming/queued messages, and emits a recovery signal. Startup reconciliation re-dispatches orphaned queued tasks, clears partial pre-running artifacts, and cancels queued children whose parent is already terminal. Recovery re-dispatch and subagent child re-dispatch are covered by tests.

12. Known limitations and roadmap

12.1 Current alpha / beta limitations

- Live model discovery is not implemented; the model registry is built-in plus user-configurable model ids.
- Write and terminal tools are intentionally not enabled until full safety boundaries ship.
- Subscription sign-in is implemented for Claude (Anthropic) and Codex/ChatGPT (OpenAI) via first-party OAuth + PKCE, behind an explicit security/ToS warning; tokens stored locally; Gemini stays API-key only.
- macOS is not supported in beta.31; Linux is source-build only.
- Cost (spentUsd) is populated only when the provider reports usage priced by Morrow.
- browser and manual criteria require external observation and are not auto-verified; they stay unverified unless evidence is attached.

! Honest gaps (from verified testing)

Known remaining engineering gaps include: no continuous onboarding→first-task thread yet proven; cancel/restart continuity needs interface-level acceptance; mid-stream reconnect/dedup after recovery; approval-after-restart path uses a direct status reset; no baseline-regression auto-block on writes; and apply-update automation is partial for the packaged build.

12.2 Roadmap stages

Morrow's first milestone is a narrow, reliable vertical slice: open the web interface; create a project and submit a task; produce a visible plan; execute one safe tool inside a scoped workspace; stream progress and evidence; persist the task across a restart; and display model, cost, files, permissions, and external data sharing. Broad integrations are added only after this slice is reliable.

Stage	Focus
Initial vertical slice	Project → task → visible plan → one scoped tool → streaming → persistence → evidence.
Multi-provider alpha	Provider-neutral runtime, presets/routing, read-only tools, truthful execution, project-isolated memory.
Durable autonomy (beta.31)	Mission continuity across context limits, browser/vision, Cortex memory, model-truth, restart recovery, extended workloads.
Planned	Desktop/web/CLI/remote experiences, write+terminal tools with safety boundaries, live model discovery, macOS.

13. Glossary

Term	Meaning
Mission	Morrow's unit of accountable work: defined success, supervised execution, evidence, review, honest grade.
Preset	A routing policy (e.g. Balanced) that resolves a provider+model and discloses the decision.
Checkpoint	A snapshot of affected file content taken before risky changes; the basis for rollback.
Evidence ledger	The persisted set of verified evidence linked to mission criteria.
Containment layer	Shared guard around read-only tools: rejects traversal, symlinks, secrets, binaries; enforces limits.
Cortex memory	Automatic, project-scoped memory built from deterministic facts and evidence-backed learnings.
MORROW_HOME	Local data root (Windows: %LOCALAPPDATA%\Morrow\data; Linux/macOS: ~/.morrow).

13.1 Ownership and licensing

Copyright © 2026 Aidan Magee. All rights reserved. No open-source license has been granted at this stage. Licensing will be decided before a public release.

13.2 Reporting issues

When reporting a problem, include the output of `morrow doctor --export` (which redacts secrets and your home prefix) along with the affected project and a description of what you expected versus what happened. Never paste API keys or credentials into issue reports.

End of manual — Morrow 0.1.0-beta.31. This document reflects the product as verified in the repository at the beta.31 release. Capability claims are backed by tests or documented evidence; limitations are called out explicitly.